



Heuristique DSATUR

Lab01

08-10-2026

Student

Mouhamed Abdallah Sakho

Student

Kimberly Beyeler

Contents

1. Introduction	3
2. Structure de données	3
3. Pseudo code	3
3.1. Correspondance avec l'algorithme 1 de l'énoncé	5
3.2. Justification de la correction	5
4. Analyse	6
4.1. Complexité temporelle	6
4.2. Complexité spatiale	7
5. Conclusion	7

1. Introduction

Le problème que nous allons résoudre dans ce laboratoire est de rédiger un pseudo code de l'heuristique de coloration séquentielle DSATUR dont la complexité temporelle et spatiale, dans le pire des cas, soit en $O(n^2)$, où n désigne le nombre de sommets du graphe traité. Le graphe $G = (V, E)$ est simple, non orienté, et fourni sous la forme d'un tableau de listes d'adjacence. Conformément à la consigne, seules des structures de données simples (tableaux et listes) sont utilisées.

2. Structure de données

Au cours du déroulement de l'algorithme, un certain nombre de données doit être connu à chaque étape :

- **Couleur** : les couleurs sont stockées dans un tableau d'entiers nommé `couleur`, de taille n , dont tous les éléments sont initialisés à -1 .
- **Degré** : les degrés initiaux des sommets sont stockés dans un tableau d'entiers nommé `degre`, de taille n , dont chaque élément est initialisé avec la taille de la liste d'adjacence du sommet correspondant.
- **Degré de saturation** : les degrés de saturation sont stockés dans un tableau d'entiers nommé `dsat`, de taille n , dont tous les éléments sont initialisés à 0 ; chaque élément représente le nombre de **couleurs différentes** présentes parmi les voisins déjà coloriés d'un sommet donné.
- **Couleurs des voisins** : les couleurs des voisins d'un sommet sont stockées dans une matrice de booléens nommée `estVoisinColorie`, de taille $n \times n$, initialisée à faux, où chaque élément indique si un sommet v possède un voisin colorié avec la couleur c .

Cette dernière structure est ce qui permet de mettre à jour le degré de saturation d'un sommet en temps constant par voisin : sans elle, il serait impossible de savoir, lorsqu'un voisin reçoit une couleur, si cette couleur est déjà connue du sommet ou non. Cela permet également d'éviter une table de hachage ou un ensemble, qui sont interdits.

Les valeurs constantes (e.g. vrai, -1) sont utilisées telles quelles dans le pseudo code.

3. Pseudo code

Le pseudo code ci-dessous détaille l'heuristique DSATUR décrite par l'algorithme 1 de l'énoncé, en explicitant les structures de données introduites à la section précédente. Il se décompose en trois phases : (1) l'initialisation des tableaux et de la matrice, (2) la coloration du premier sommet traitée séparément, puisque tous les degrés de saturation valent alors 0 et ne permettent donc pas encore de distinguer les sommets entre eux, et (3) la boucle principale qui colorie les $n - 1$ sommets restants en sélectionnant à chaque itération le sommet non colorié de plus grande saturation. La correspondance précise entre ce pseudocode et l'algorithme 1 de l'énoncé est détaillée juste après celui-ci, et sa correction est justifiée à la section suivante.

Algorithm 1: Heuristique DSATUR (version détaillée)

```

1:  procedure DSATUR(Adj, n)
2:    ▷ Phase 1 : Initialisation,  $\Theta(n^2)$ 
3:    for  $v \leftarrow 0$  à  $n - 1$  do
4:      couleurv  $\leftarrow -1$ 
5:      dsatv  $\leftarrow 0$ 
6:      degrev  $\leftarrow$  taille(Adjv)
7:      for  $c \leftarrow 0$  à  $n - 1$  do
8:        estVoisinColoriev,c  $\leftarrow$  faux
9:      end
10:   end
11:   nbColories  $\leftarrow 0$ 
12:   ▷ Phase 2 : Coloration du premier sommet (plus grand degré),  $\Theta(n)$ 
13:    $u \leftarrow 0$ 
14:   for  $v \leftarrow 1$  à  $n - 1$  do
15:     if degrev > degreu then
16:        $u \leftarrow v$                                 ▷ départage : le plus petit indice est conservé
17:     end
18:   end
19:   couleuru  $\leftarrow 0$ 
20:   nbColories  $\leftarrow$  nbColories + 1
21:   for  $w \in \text{Adj}_u$  do
22:     estVoisinColoriew,0  $\leftarrow$  vrai
23:     dsatw  $\leftarrow$  dsatw + 1
24:   end
25:   ▷ Phase 3 : Boucle principale, exactement  $n - 1$  itérations,  $\mathcal{O}(n^2)$  au total
26:   while nbColories <  $n$  do
27:     ▷ 3.a : Sélection du sommet non colorié de plus grand degré de saturation
28:      $u \leftarrow -1$ 
29:     for  $v \leftarrow 0$  à  $n - 1$  do
30:       if couleurv =  $-1$  then
31:         if  $u = -1$  ou dsatv > dsatu ou (dsatv = dsatu et degrev > degreu)
32:           then
33:              $u \leftarrow v$                                 ▷ double égalité : plus petit indice conservé
34:           end
35:         end
36:       ▷ 3.b : Plus petite couleur disponible pour u
37:        $c \leftarrow 0$ 
38:       while estVoisinColorieu,c do
39:          $c \leftarrow c + 1$ 
40:       end
41:       couleuru  $\leftarrow c$ 
42:       nbColories  $\leftarrow$  nbColories + 1
43:       ▷ 3.c : Mise à jour du degré de saturation des voisins non coloriés
44:       for  $w \in \text{Adj}_u$  do
45:         if couleurw =  $-1$  et non estVoisinColoriew,c then
46:           estVoisinColoriew,c  $\leftarrow$  vrai
47:           dsatw  $\leftarrow$  dsatw + 1
48:         end
49:       end
50:   end

```

```

51:  return couleur
52:  end

```

3.1. Correspondance avec l'algorithme 1 de l'énoncé

- **Lignes 4 et 5 de l'algorithme 1 (premier sommet).** Correspondent à la Phase 2 : comme toutes les saturations valent 0 au début, elles ne permettent pas encore de distinguer les sommets entre eux, d'où la sélection par degré initial seul ; le sommet retenu reçoit la couleur 0, puis la saturation de ses voisins est mise à jour.
- **Ligne 7 (choix du sommet).** Correspond au bloc 3.a : sélection par saturation maximale, puis par degré initial maximal en cas d'égalité, le plus petit indice départageant les égalités restantes.
- **Ligne 8 (couleur).** Correspond au bloc 3.b : recherche de la plus petite couleur c telle que $\text{estVoisinColorie}_{u,c}$ vaut faux.
- **Ligne 9 (mise à jour).** Correspond au bloc 3.c : seuls les voisins non coloriés de u sont mis à jour, et la saturation d'un voisin n'augmente que si la couleur attribuée lui est nouvelle.

3.2. Justification de la correction

Les trois invariants suivants sont vérifiés au début de chaque itération de la boucle principale (Phase 3).

1. **Matrice.** Pour tout sommet non colorié w et toute couleur c , $\text{estVoisinColorie}_{w,c}$ vaut vrai si et seulement si w possède **au moins un** voisin colorié de couleur c .
2. **Saturation.** Pour tout sommet non colorié w , dsat_w est égal au nombre de couleurs c pour lesquelles $\text{estVoisinColorie}_{w,c}$ vaut vrai, exactement la définition du degré de saturation donnée par l'énoncé.
3. **Coloration propre.** Deux sommets adjacents coloriés n'ont jamais la même couleur.

Ces invariants se maintiennent de la manière suivante.

- Ils sont vérifiés à l'issue des Phases 1 et 2 : aucun sommet n'est colorié après la Phase 1, et la mise à jour des voisins du premier sommet en Phase 2 suit exactement le même mécanisme que le bloc 3.c.
- Lors de la mise à jour (3.c), un voisin non colorié w de u ne voit $\text{estVoisinColorie}_{w,c}$ passer à vrai, et dsat_w s'incrémenter, que si cette entrée était fausse auparavant, c'est-à-dire si aucun autre voisin colorié de w n'avait déjà la couleur c . Les invariants 1 et 2 restent donc vrais.
- L'invariant 3 découle de l'invariant 1 : la couleur c attribuée en 3.b est, par construction, absente des voisins déjà coloriés de u .

Absence de dépassement de la matrice. Au moment de la recherche en 3.b, la ligne de u dans la matrice contient au plus $\text{degre}_u \leq n - 1$ entrées vraies (une par voisin colorié au maximum), pour n entrées au total ; elle contient donc toujours au moins une entrée fausse, atteinte après au plus $\text{degre}_u + 1 \leq n$ essais. L'indice c reste ainsi

toujours dans $\{0, \dots, n-1\}$, ce qui justifie a posteriori qu'une matrice $n \times n$ suffit, comme annoncé à la section 2.

Couleurs consécutives. Une couleur $c > 0$ n'est attribuée que si les couleurs $0, \dots, c-1$ sont déjà présentes chez un voisin colorié de u , donc déjà utilisées ailleurs dans le graphe. Les couleurs utilisées forment ainsi toujours un ensemble $\{0, 1, \dots, k\}$ sans trou, conformément à l'exigence de l'énoncé.

4. Analyse

On note $n = |V|$ le nombre de sommets et $m = |E|$ le nombre d'arêtes du graphe. Comme le graphe G est simple, chaque arête relie deux sommets distincts et ne peut être comptée qu'une seule fois ; le nombre d'arêtes m est donc borné par le nombre de paires de sommets distincts que l'on peut former parmi les n sommets du graphe, soit $m \leq \binom{n}{2} = \frac{n(n-1)}{2}$. Ce majorant étant de l'ordre de n^2 , on en déduit que $m = O(n^2)$, quel que soit le nombre d'arêtes effectivement présentes dans le graphe.

4.1. Complexité temporelle

Phase 1 : Initialisation. La boucle externe (sur v) s'exécute n fois ; pour chaque v , la boucle interne sur les couleurs s'exécute également n fois, chaque itération ne coûtant que $O(1)$ (simple affectation dans un tableau). Le coût total de cette phase est donc $\Theta(n) \cdot \Theta(n) = \Theta(n^2)$.

Phase 2 : Coloration du premier sommet. La recherche du sommet de plus grand degré est un parcours linéaire en $\Theta(n)$, et la mise à jour de ses voisins coûte $O(\text{degre}(u)) = O(n)$ dans le pire cas. Le coût total de cette phase est donc $\Theta(n)$, négligeable devant la Phase 1.

Phase 3 : Boucle principale. Cette boucle colorie exactement un sommet par itération et s'arrête lorsque tous les sommets sont coloriés ; comme un sommet a déjà été colorié en Phase 2, elle s'exécute exactement $n-1$ fois. À chaque itération, trois étapes interviennent :

- **3.a : Sélection du sommet.** Un parcours linéaire des n sommets, chaque test étant $O(1)$: coût $\Theta(n)$ par itération.
- **3.b : Recherche de la plus petite couleur libre.** Le nombre de couleurs présentes parmi les voisins coloriés de u est au plus $\text{degre}(u)$ (chaque voisin colorié ne peut contribuer qu'à une seule case vraie de la matrice). Il existe donc toujours un indice $c \leq \text{degre}(u) \leq n-1$ pour lequel `estVoisinColorie` vaut faux, ce qui borne cette recherche à $O(\text{degre}(u)) \subseteq O(n)$ par itération.
- **3.c : Mise à jour des voisins.** Un parcours des voisins de u , en $O(\text{degre}(u))$.

Le coût d'une itération est donc $O(n)$, ce qui donne, cumulé sur les $n-1$ itérations :

$$\sum_{k=1}^{n-1} O(n) = O(n^2).$$

On peut affiner ce résultat en observant que chaque sommet n'est sélectionné comme u qu'une seule fois durant toute l'exécution (Phase 2 incluse) : la somme, sur toute l'exécution, des coûts des blocs 3.b et 3.c vaut donc $\sum_{v \in V} O(\text{degre}(v)) = O(2m) = O(m) \subseteq O(n^2)$ (lemme des poignées de main), ce qui confirme la borne obtenue ci-dessus.

Bilan. En sommant les trois phases, $\Theta(n^2) + \Theta(n) + O(n^2) = \Theta(n^2)$. Cette borne est atteinte, et non simplement majorée : même sur un graphe sans arête ($m = 0$), l'initialisation de la matrice coûte déjà $\Theta(n^2)$ à elle seule, tout comme le seul balayage linéaire de sélection (3.a), répété $n - 1$ fois indépendamment du nombre d'arêtes. La complexité temporelle de l'algorithme est donc $\Theta(n^2)$ dans le pire des cas.

4.2. Complexité spatiale

L'espace auxiliaire utilisé par l'algorithme se décompose comme suit :

Structure	Taille	Complexité
couleur, degre, dsat	$3n$ entiers	$O(n)$
estVoisinColorie	$n \times n$ booléens	$O(n^2)$
Variables scalaires ($u, v, w, c, \text{nbColories}$)	–	$O(1)$

L'espace total est donc $O(n) + O(n^2) + O(1) = O(n^2)$, dominé par la matrice `estVoisinColorie`. Cette matrice est indispensable : c'est elle qui permet la mise à jour en $O(1)$ par voisin sans recourir à une structure complexe telle qu'une table de hachage, conformément à la contrainte de l'énoncé. Comme pour le temps, cette borne est atteinte dès que $n \geq 1$, indépendamment du nombre d'arêtes : la complexité spatiale est donc également $\Theta(n^2)$ dans le pire des cas.

5. Conclusion

Le pseudo code proposé répond à l'objectif fixé par l'énoncé : obtenir une implémentation de l'heuristique DSATUR dont les complexités temporelle et spatiale sont toutes deux en $O(n^2)$ dans le pire des cas, en n'utilisant que des structures de données simples. Deux choix de conception permettent d'atteindre cette borne :

- la matrice booléenne `estVoisinColorie`, qui donne un test de présence d'une couleur en $O(1)$ et une mise à jour du degré de saturation en $O(1)$ par voisin, pour un coût cumulé en $O(m)$ sur toute l'exécution ;
- la recherche linéaire du sommet à colorier à chaque itération, qui coûte $O(n)$ et rend inutile le recours à un tas, structure explicitement interdite par l'énoncé.

L'analyse montre que la borne $\Theta(n^2)$ est atteinte indépendamment du nombre d'arêtes du graphe, ce qui confirme que cette complexité est bien la complexité réelle de l'algorithme, et non une simple majoration.